

Decision Trees, RFs, and DNNs

Machine Learning and Security

Fabio De Gaspari
Dorjan Hitaj

Outline

- Decision Trees
- Decision Forests: Random Forest
- Deep Neural Networks

Decision Trees

Decision Trees

Hierarchical collection of conditions (or splits, tests) where each node represents a prediction

- conditions are based on features of the data and decision criteria are trained
  - conditions can be axis-aligned if they involve a single feature, or oblique if they involve multiple

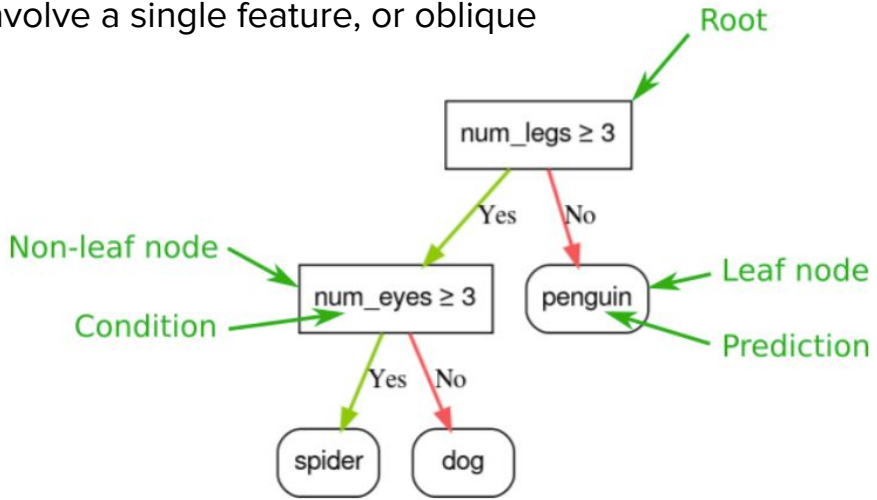


Image credit: Google

Decision Trees

- Binary decision trees have only binary conditions
  - generally used by decision forests (e.g., Random Forests - RF)
- Non-binary decision trees have variable number of conditions
  - too many conditions can lead to overfitting

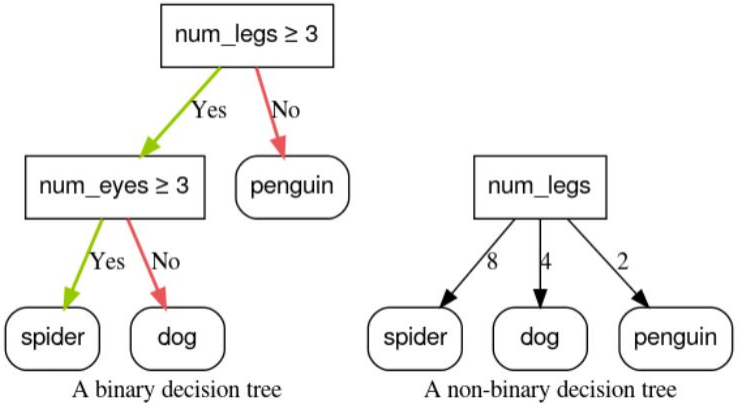


Image credit: Google

Decision Trees: Training

Optimal training is NP-hard problem.

- Training follows heuristic methods, e.g., greedy divide and conquer
- Starting from a root node, the tree is recursively grown
  - at each node, possible conditions are evaluated and the “best” one is selected
  - various metrics can be used to select which one is “best”
- Depending on the type of input features, possible conditions for a node is generally infinite
  - heuristic approaches to identify suitable conditions are used
- We stop when a newly-created node has only (or almost-only) samples from the same class: it has high ***purity***
  - we want leaf nodes with high purity

Decision Trees: Choosing a Condition's Feature

Entropy can be used as a measure of purity:

- can be thought of as a measure of disorder

  $-\sum_{0}^{C} y_i log_2(y_i)$ where y_i is the fraction of samples of class i

  - maximum when samples are mixed (e.g., 50/50 for two-class problem)
  - minimum when all samples are all from the same class

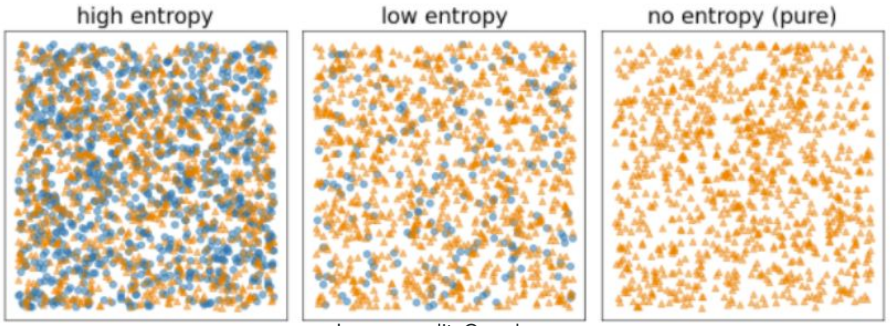


Image credit: Google

Decision Trees: Choosing a Condition's Feature

Information Gain computes the expected reduction in entropy obtained by partitioning the samples according to a given feature and value

$$G(D, f) = entropy(D) - \sum_{v \in values(f)} \frac{|D_v|}{|D|} entropy(D_v)$$

where:

- *D* is the set of samples at a node of the tree
- *f* is one of the features of the data
- D_v is the subset of *D* having value *v* for feature *f*
  - e.g., considering color as feature *f*, D_v is the subset for which *v=red*

Essentially, the entropy of each possible resulting set, weighted by its relative size

Decision Trees: Choosing a Condition's Feature

Information Gain computes the expected reduction in entropy obtained by partitioning the samples according to a given feature

$$G(D, f) = entropy(D) - \sum_{v \in values(f)} \frac{|D_v|}{|D|} entropy(D_v)$$

Given a feature for a condition, how are the values for the split chosen?

- straightforward for discrete features: test them, chose largest *G*
- continuous features must be discretized beforehand, then largest *G*

Recursively Apply until no selection features are left

Random Forest

Ensemble Methods

Ensemble methods combine multiple classifiers into a single classifier

- each classifier learns a target function, and their individual predictions are combined to classify new examples
- typically improve generalization performance compared to single models
  - combining multiple *independent* and *diverse* decisions reduces variance and the final error rate

Random Forest

Ensemble method combining multiple decision tree classifiers

- Combination of bootstrapped decision trees
  - for each tree, randomly sample a subset of instances from the training data
  - train an unpruned decision tree on the sampled data
- For each tree, at each decision node the best feature is chosen from a random sample of *m* features out of the total number of features
  - the best split is computed based on these *m* features through, e.g., information gain
  - this is called the ***random vector method***

Random Forest

Overall, RF:

- Randomly samples *k* subsets of training data with replacement (bootstrapping)
- For each subset:
  - train a decision tree via the random vector method
  - estimate the error of the decision tree using the remaining examples
- Aggregate the prediction of each individual tree to make a final prediction
  - typically uses majority voting, but other approaches can be used (e.g., weighted)

Why Random Forest

RF has several advantages as a learner:

- Bootstrapping and random vector promote independence among DTs
  - key requirement for ensambles to perform well
  - decreases chances of overfitting when increasing the number of DTs in the forest
- Reduced dependence of prediction on a specific training sample
- Fairly computationally inexpensive compared to complex neural networks, while retaining typically good performance for many tasks

Deep Neural Networks

Deep Neural Networks

Deep Neural Network (DNN) is an umbrella term covering many types of networks

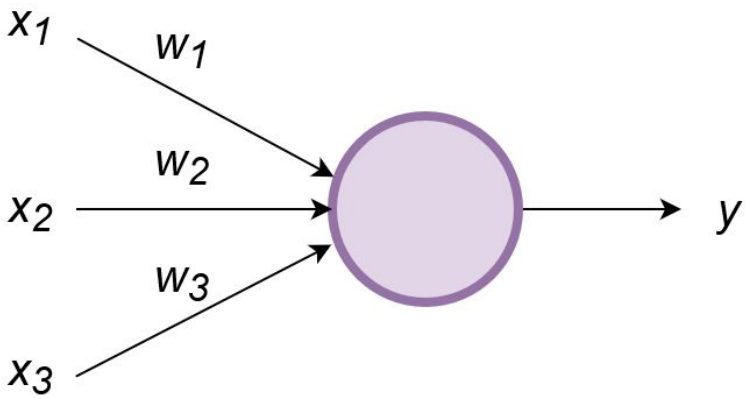
- Typically refers to any neural network with many *layers*

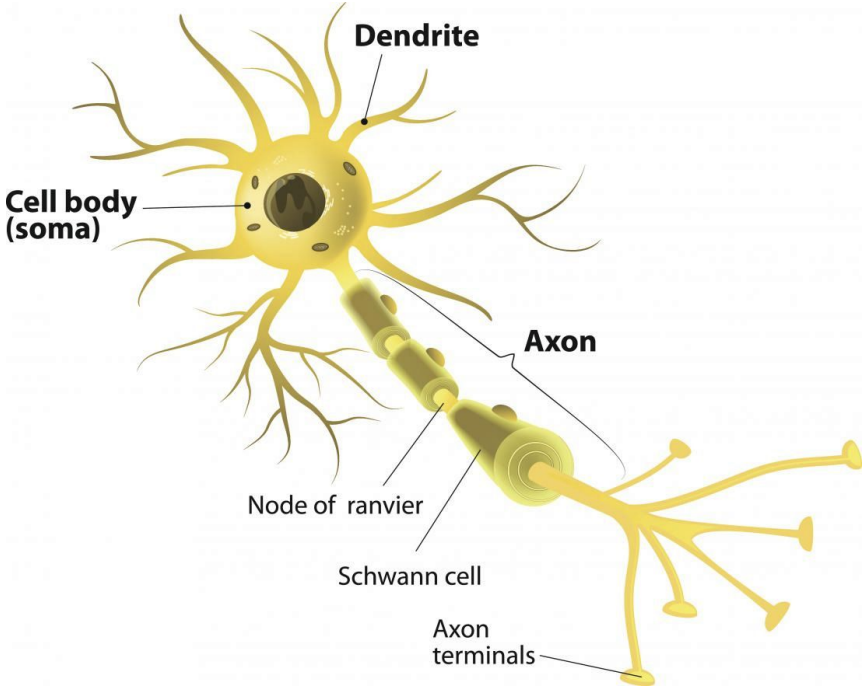
Many different types, based on what we need to learn

- Deep Feedforward Networks
- Convolutional Neural Networks
- Recurrent Neural Networks
- Generative Adversarial Networks
- Transformer-based Neural Networks
- ...

Deep Neural Networks

Why the name Neural Networks?





Deep Feedforward Networks

DNNs are functions approximating other (unknown) functions

- given a function defining a mapping between an input x and a category y:

$$y = f^*(x)$$

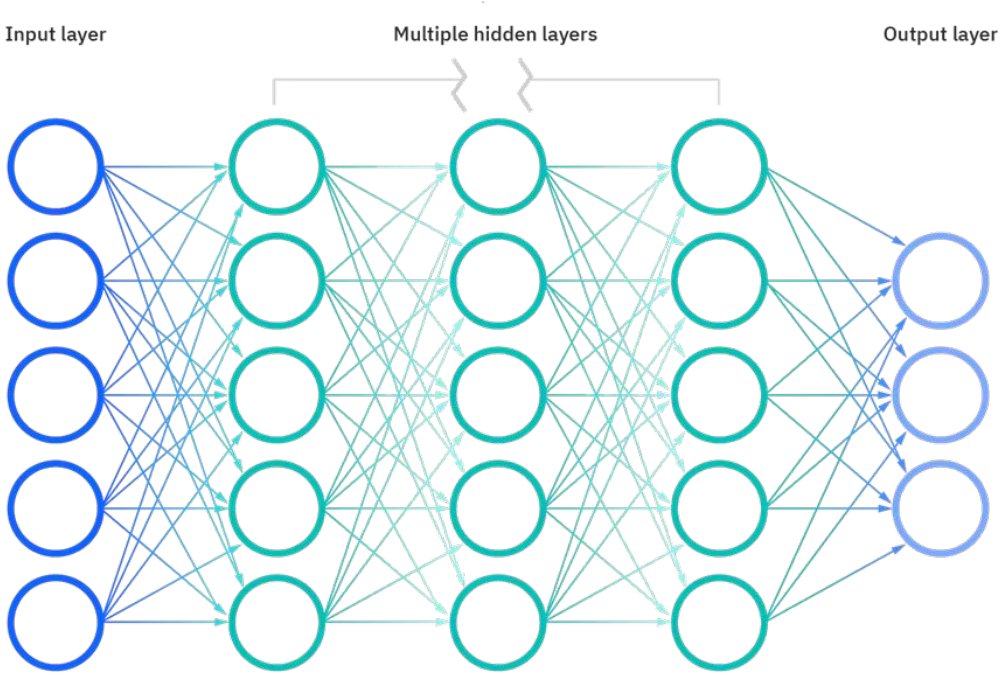
- a DNN defines a mapping:

$$\hat{y} = f(x; \theta)$$

The DNN learns the value of parameters θ that result in the best approximation of f^*

Deep Feedforward Networks

Why feedforward?



Deep Feedforward Networks

They are called networks because they typically compose together several functions

- The network is a directed acyclic graph describing how the functions are composed
- The composition of the functions is the neural network function

For example, for a network of three functions f^1, f^2, f^3:

$$f(x) = f^3(f^2(f^1(x)))$$

Deep Feedforward Networks

For example, for a network of three functions f^1, f^2, f^3:

$$f(x) = f^3(f^2(f^1(x)))$$

- Here, the network is the function f
- Each of the $f^\#$ functions represents one *layer* of the network
- The length of the composition is the depth of the network
  - Deep networks are, therefore, compositions of many functions

During training, we 'teach' $f(x)$ to get closer and closer to $f^*(x)$

Deep Feedforward Networks

But how can we get closer $f^*(x)$, if the function itself is unknown?

- We use large amounts of training data (dataset)
- Each datapoint x_i in the dataset is accompanied by a *label* y_i
- The label indicates the result of the application of f^* to x:

$$y_i = f^*(x_i)$$

- For each x_i, the goal of the training is to get the output layer to produce a value as close to its respective y_i as possible

Deep Feedforward Networks

What about the hidden layers?

- The training data only specifies input and output
- The hidden layers are modified by the learning algorithm to achieve a good approximation of f^*

Deep Feedforward Networks - Main Idea

The main idea behind feedforward NN is to apply a *linear* model to a *nonlinear* transformation of the input x:

- deep learning introduces a nonlinear transformation ϕ: R^n –> R^n

$$\hat{y} = f(x; \theta; w) = \phi(x; \theta)^T w$$

- and learns both the transformation and the parameters w for the linear mapping, similarly to what we saw for linear regression

$$\hat{y} = w^T x + b$$

Deep Feedforward Networks - Main Idea

$$\hat{y} = f(x; \theta; w) = \phi(x; \theta)^T w$$

NNs learn:

- The parameters θ, defining the nonlinear function ϕ (although often fixed)
- The parameters *w*, that map from $\phi(x)$ to the desired output

This example defines a deep feedforward network with one hidden layer: ϕ

- ϕ can be thought of as a *feature extractor*, that outputs a “good” representation of *x* for the task
  - it maps an input into an intermediate feature space learned by the network
- *w* applies a linear transformation from the learned representation to the desired output

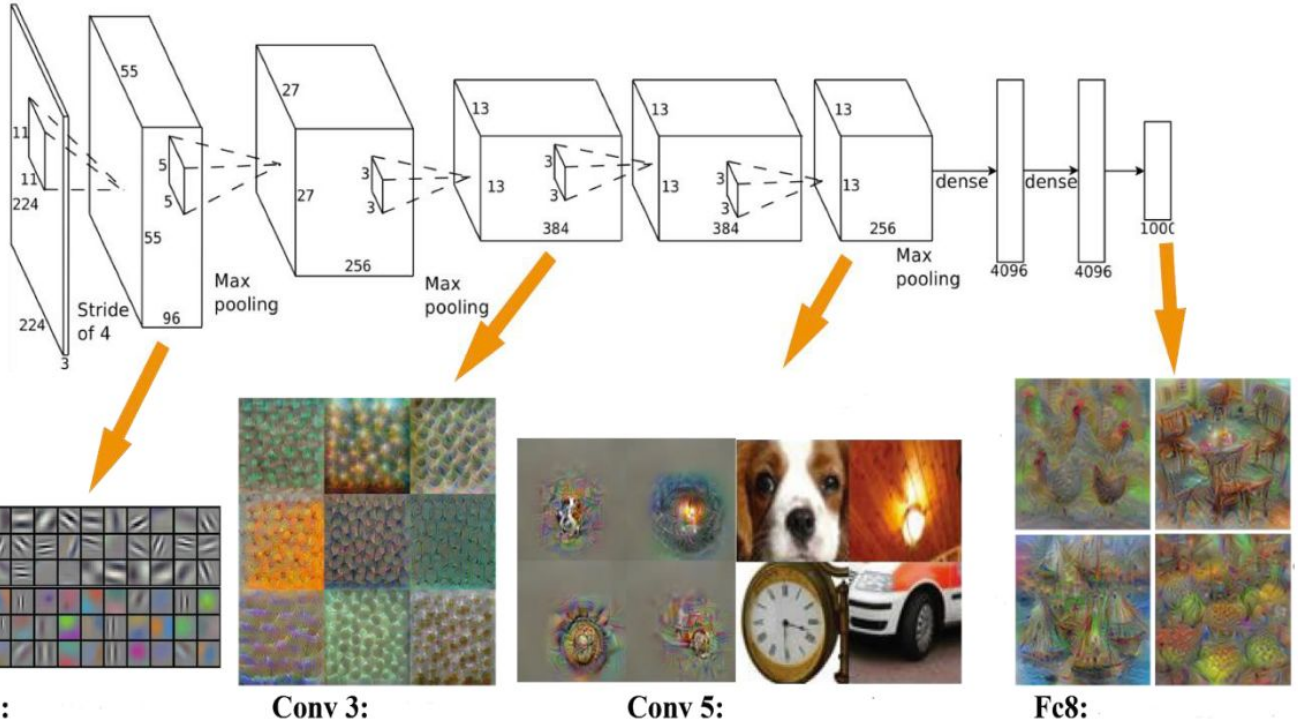
Deep Feedforward Networks - Main Idea

NNs le

- Th
- Th

This ex

- ϕ (
  re
  -
- w
  desired output



Deep Feedforward Networks - Pros and Cons

$$\hat{y} = f(x; \theta; w) = \phi(x; \theta)^T w$$

Cons:

- giving up convexity of the training problem - no closed form solution

Pros:

- can learn ϕ and *w* from a broad family of functions
- we can tweak the family of functions ϕ that the model can learn (or fix it) to help generalization

Typically, NNs use an affine transformation with learnable parameters *w* followed by a fixed, nonlinear function ϕ

Deep Feedforward Networks - Notation

$$\hat{y} = f(x; \theta; w) = \phi(x; \theta)^T w$$

Until here, we made the distinction between θ, ϕ and w.

- going forward, we will simplify the notation dropping the distinction between weights of the linear function and the nonlinear function:

$$\hat{y} = f(x; \theta) = f(x; W, \phi)$$

where θ indicates all the parameters that define the function learned by the network (weights and nonlinear functions ϕ)

DNNs
Understanding ϕ

Understanding ϕ

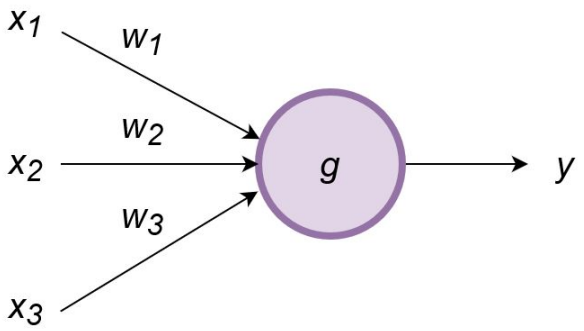
A simple example: a music festival you would like to go to, based on three factors

- weather: it's in the open, you don't like to get wet
- friends: are they coming with you?
- transport: how annoying is it to get there?

There are three factors, each more or less important, based on which you can make a decision, depending on how much you like music

Understanding ϕ

- weather: it's in the open, you don't like to get wet (x_1)
- friends: are they coming with you? (x_2)
- transport: how annoying is it to get there? (x_3)

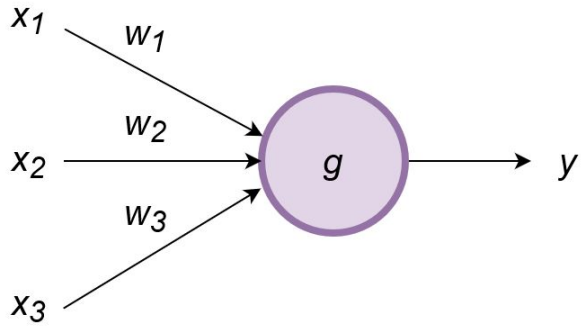


Three factors (x_1, x_2, x_3) – more or less important (w_1, w_2, w_3) – based on which you can make a decision (y), depending on a function of how much you like music (*g)*

Understanding ϕ

$$g = \begin{cases} 1 & if \sum_i x_i w_i \geq threshold \\ 0 & if \sum_i x_i w_i < threshold \end{cases} =$$

$$= \begin{cases} 1 & if\ wx + b \geq 0 \\ 0 & if\ wx + b < 0 \end{cases}$$



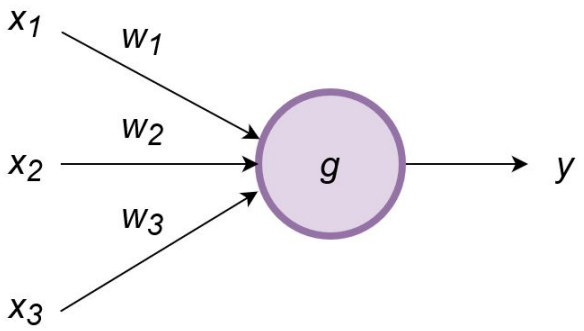
- varying the parameters *w* changes the decision making model, giving more or less weight to one of the three factors *x* (features)
- *g* is called the ***activation function***, and defines the output of the neuron. Here, it's a simple threshold
- in NNs, *g* is a nonlinear function of the inputs x_i and weights w_i
- together, *g* and *w* correspond to the parameters Θ we discussed earlier

Understanding ϕ - Activation Functions

The activation function (g) defines the behavior of the function learned by the network. There exist many types of activation functions:

- linear: $g(z) = \lambda z + b$
- sigmoid: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU: $g(z) = max\{0, z\}$
- Softmax: $g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$
- ...

where $z = w^T x$



Understanding ϕ - Activation Functions

linear: $g(z) = \lambda z$

- this is what we were using in linear regression
- given $z = w^T x + b \rightarrow \hat{y} = g(z) = \lambda(w^T x + b)$
  - with $w, x \in R^n$; $\lambda, b \in R$
  - the bias b is an additional parameter of the neuron, together with w
- however, if we use a linear ϕ, the whole composition becomes a linear function
- meaning we can only learn linear approximations of f^*

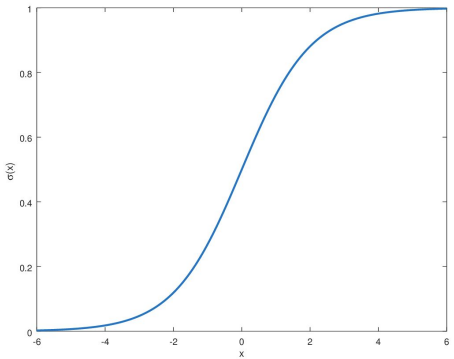
not good, not used

Understanding ϕ - Activation Functions

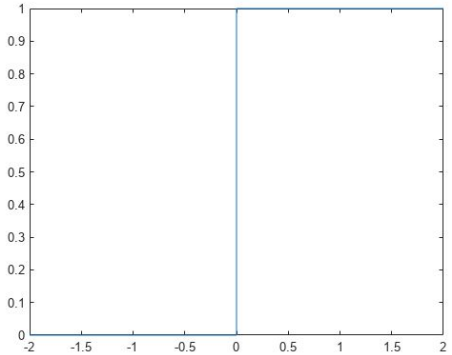
sigmoid: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$

- given $z = w^T x$; $\hat{y} = g(z) = \sigma(z) = \frac{1}{1+exp(-w^T x - b)}$

Sigmoid



Step Function



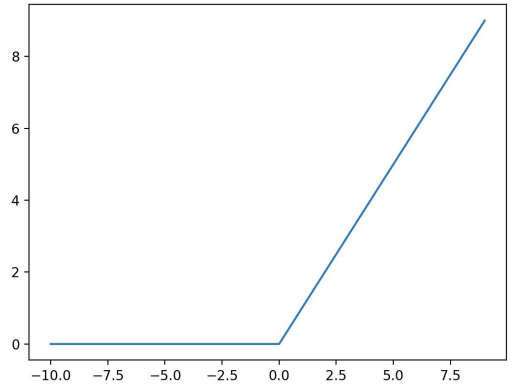
sigmoid smoothes the step activation function

- fundamental for learning, as we will see when we discuss backpropagation

Understanding ϕ - Activation Functions

ReLU: $g(z) = max\{0, z\}$

- given $z = w^T x$; $g(z) = max\{0, w^T x\}$



- most used in feedforward NNs. Piecewise linear function, maintains useful properties of linear functions that help gradient-based learning

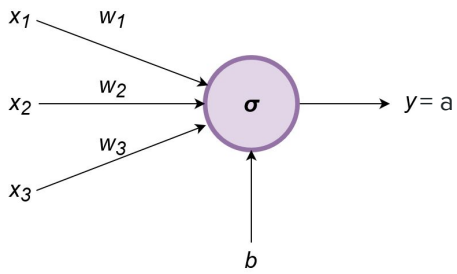
Understanding ϕ - Activation Functions

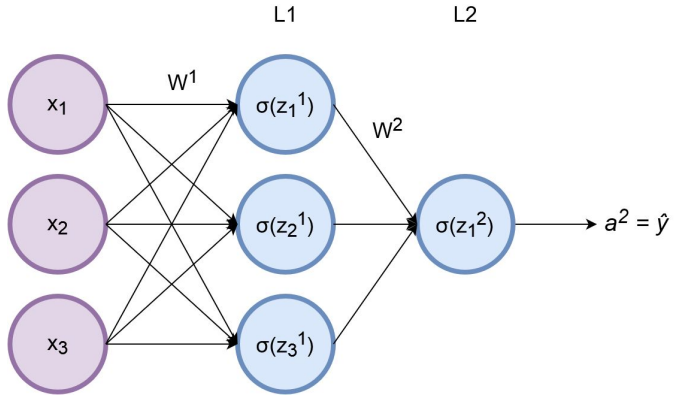
softmax: $g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- given $z = w^T x_i$; $g(z_i)$ $\frac{e^{w^T x_i}}{\sum_j e^{w^T x_j}}$
- multivariate: every output depends on every input in that layer; hard to plot
- it produces an output vector normalized in [0, 1]
  - can be thought of as a probability distribution, where each element of the vector is the probability for that element
  - useful for e.g., multi-class classification tasks

DNNs
Forward and Backward Propagation

Stacking Multiple Layers





$z = W^T x + b$; $\hat{y} = a = \sigma(z)$;

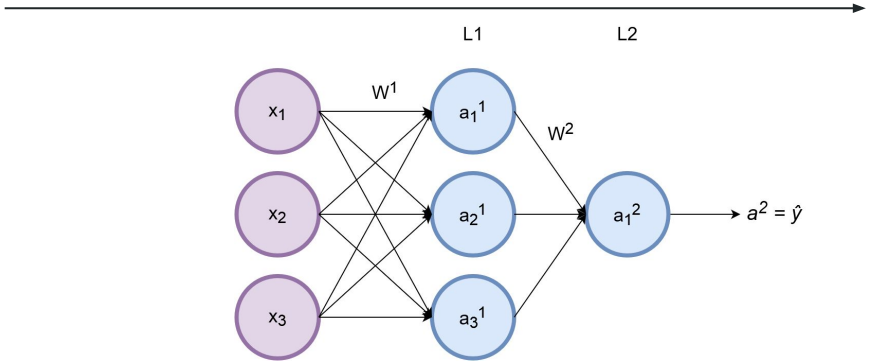
the loss function: $L(\hat{y}, y) = L(a, y)$

$z^1 = W^1 x + b^1$; $a^1 = \sigma(z^1)$; $z^2 = W^2 a^1 + b^2$; $\hat{y} = a^2 = \sigma(z^2)$;

the loss function: $L(\hat{y}, y) = L(a^2, y)$

Note: bias b not shown in picture

DNN: Forward Propagation



$W^1 \in R^{3,3}; b^1 \in R^{1,3}; W^2 \in R^{3,1}, b^2 \in R^{1,1}$

$$z^1 = W^1x + b^1; a^1 = \sigma(z^1); \rightarrow \begin{cases} z_1^1 = w_1^1x_1 + b_1^1; a_1^1 = \sigma(z_1^1); \\ z_2^1 = w_2^1x_2 + b_2^1; a_2^1 = \sigma(z_2^1); \\ z_3^1 = w_3^1x_3 + b_3^1; a_3^1 = \sigma(z_3^1); \end{cases}$$

Gradient Descent

Recall that the derivative defines the rate of change of *f* with respect to a variable

- it can be used to scale a small change in the input to obtain the corresponding small change in the output

$$f(x + \epsilon) \cong f(x) + \epsilon \frac{\mathrm{d}}{\mathrm{d}x} f(x)$$

Recall that we want to minimize the cost function *J* over the whole dataset

- with the *loss function L* defining the error for each individual prediction $\hat{y}_i$

$$J(\theta) = \frac{1}{m} \sum_{1}^{m} L(\hat{y}, y) = \frac{1}{m} \sum_{1}^{m} L(f(x; \theta), y)$$

How do we train our network? What do we optimize?

Gradient Descent

$$J(\theta) = \frac{1}{m}\sum_{1}^{m} L(\hat{y}, y) = \frac{1}{m}\sum_{1}^{m} L(f(x;\theta), y)$$

Gradient Descent (GD) minimizes the cost *J* by changing the network parameters θ iteratively

For each example in the dataset

- calculate the derivative of *L* w.r.t. θ
- iteratively update θ to minimize the loss (and cost) functions

$$\theta = \theta - \alpha\frac{\partial}{\partial\theta}L(f(x;\theta), y) = \theta - \alpha\nabla_\theta L(\hat{y}, y)$$

- in practice, gradient is computed over batches of samples, as we will discuss later

Gradient Descent

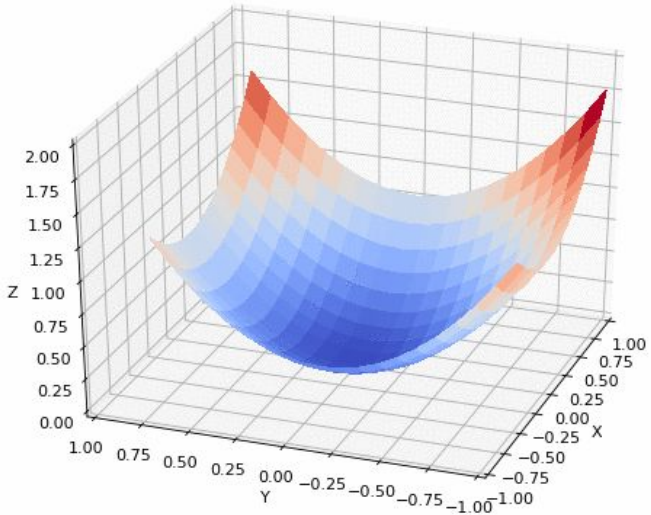


image credits to https://blog.paperspace.com/intro-to-optimization-in-deep-learning-gradient-descent/

Gradient Descent

$$\theta = \theta - \alpha \frac{\partial}{\partial \theta} L(f(x;\theta), y) = \theta - \alpha \nabla_\theta L(\hat{y}, y)$$

In our Feedforward DNN example, the parameters θ are:

- multiple weight matrices *W*
- multiple bias vectors *b*

So, for each layer of the network, the optimization becomes

$$dW^i = \frac{\partial}{\partial W^i} J; db^i = \frac{\partial}{\partial b^i} J$$

$$W^i = W^i - \alpha dW^i; b^i = b^i - \alpha db^i$$

Gradient Descent

$$\theta = \theta - \alpha \frac{\partial}{\partial \theta} L(f(x;\theta), y) = \theta - \alpha \nabla_\theta L(\hat{y}, y)$$

In our Feedforward DNN example, the parameters θ are:

- multiple weigh
- multiple bias vectors b

Careful! in papers loss and cost are sometimes used interchangeably

So, for each layer of the network, the optimization becomes

$$dW^i = \frac{\partial}{\partial W^i} J; db^i = \frac{\partial}{\partial b^i} J$$

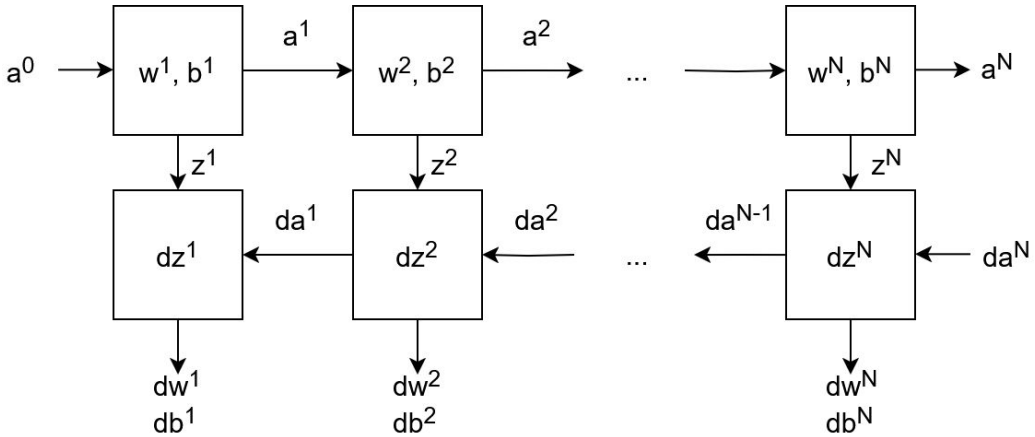
$$W^i = W^i - \alpha dW^i; b^i = b^i - \alpha db^i$$

Gradient Descent: Backpropagation

We need to update weights and biases on all layers to train the DNN:

Forward Pass

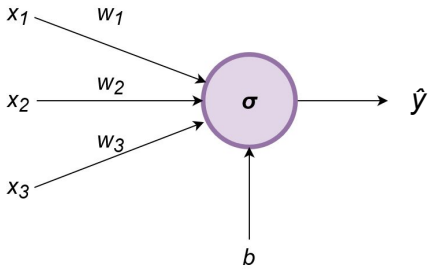
Backward Pass
(Backpropagation)



$$W^i = W^i - \alpha dW^i; b^i = b^i - \alpha db^i$$

Gradient Descent: Backpropagation

Use the chain rule to rewrite the derivative w.r.t. *W*. For a single neuron:



$z = W^T x + b$; $\hat{y} = a = \sigma(z)$;

$L(\hat{y}, y) = L(\hat{y}, y)$

$$\frac{\partial}{\partial w} L = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}$$

We need to update weights and biases on all layers to train the DNN:

- we'll do this by 2 steps:
  a. equation for error in the output layer wrt z (dz^N)
  b. equation for error in layer *i* wrt a^i (da^i) in terms of error in layer i+1 wrt z (dz^{i+1})

Backpropagation : Error in output layer

for the last layer (output), the rate of change of the loss wrt z is the first part: $\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}$

- for a network with N layers, $a^i = g(z^i)$, where $a^i \in R^{n-i}$ are the activations at layer i
  - Therefore $\hat{y} = a^N$
- The error in the last layer wrt. z is

$$dz^N = \nabla_{z^N} J = \nabla_{a^N} J \frac{\partial a^N}{\partial z^N}$$

where

- the first term is the gradient of the loss J w.r.t. the output of the network (a^N)
- the second term is the Jacobian matrix of the function g: $a^N = g(z^N)$. In our example, $g = \sigma$

Backpropagation : Error in output layer

$$dz^N = \nabla_{z^N} J = \nabla_{a^N} J \frac{\partial a^N}{\partial z^N}$$

- the first term is the gradient of the loss J wrt. the output of the network (a^N)
- the second term is the Jacobian matrix of the function *g*: $a^N = g(z^N)$

The Jacobian matrix is an extension of the gradient for functions whose input and output are both vectors *g:* $R^n \rightarrow R^m$

- it's the matrix containing all of the partial derivatives of g

$$\boldsymbol{Jac} \in R^{n,m}; \quad \boldsymbol{Jac}_{i,j} = \frac{\partial}{\partial z_j} g(z)_i = \frac{\partial a_i}{\partial z_j}$$

Backpropagation : Error in output layer

$$dz^N = \nabla_{z^N} J = \nabla_{a^N} J \frac{\partial a^N}{\partial z^N} \quad \text{(eq. 1)}$$

Eq. 1 is computing:

- the rate of change of the error (loss) wrt. the activations in the last layer
- the rate of change of the activation function wrt. the pre-activation values

Backpropagation : Error in i From Error in i+1

Recall that $z^{i+1} = W^{i+1}a^i + b^{i+1}$

- we want to find $\nabla_{a^i} J = \frac{\partial}{\partial a^i} J = \frac{\partial J}{\partial z^{i+1}} \frac{\partial z^{i+1}}{\partial a^i} = dz^{i+1} \frac{\partial z^{i+1}}{\partial a^i}$

It can be shown that this is equivalent to

$$da^i = \nabla_{a^i} J = (W^{i+1})^T dz^{i+1} \qquad \text{(eq. 2)}$$

Intuitively, it indicates that the error for layer i changes proportionally to the error in i+1, multiplied by the weights between i and i+1

Backpropagation

Together, eq. 1 and eq. 2 allow us to propagate the error backward, from the output layer to the first hidden layer

- from each dz^i, we can easily calculate db^i and dW^i:

$$dW^i = dz^i (a^{i-1})^T \qquad \text{(eq. 3)}$$

$$db^i = dz^i \qquad \text{(eq. 4)}$$

Backpropagation

Summarizing:

$$dz^N = \nabla_{z^N} J = \nabla_{a^N} J \frac{\partial a^N}{\partial z^N} \qquad \text{(eq. 1)}$$

$$da^i = \nabla_{a^i} J = (W^{i+1})^T dz^{i+1} \qquad \text{(eq. 2)}$$

$$dW^i = dz^i (a^{i-1})^T \qquad \text{(eq. 3)}$$

$$db^i = dz^i \qquad \text{(eq. 4)}$$

DNN
Stochastic Gradient Descent and Loss Functions

Stochastic Gradient Descent

We've seen that with SGD we update the weights after calculating the gradient over all examples

- typically, it is not feasible to use the whole dataset for each step of SGD
  - training would never converge for large datasets with hundreds of thousands or millions of samples

The solution is *Stochastic* Gradient Descent

Stochastic Gradient Descent

Rather than computing dW and db using all our samples, we:

1. take a *random subset* of the training data (called **mini-batch**) of size *m*
2. compute $dW^i = \frac{\partial}{\partial W^i} J; db^i = \frac{\partial}{\partial b^i} J$ for each sample in the mini-batch
3. update the weights and biases

$$W = W - \frac{\alpha}{m} dW; b = b - \frac{\alpha}{m} db$$

4. keep iterating until we sample the whole dataset
5. repeat 1-4 for the desired number of epochs

Common Loss Functions

There are many loss functions, used for different purposes

- Mean Squared Error (MSE)
- Binary Cross-Entropy Loss
- Cross-Entropy Loss
- Contrastive Loss / Triplet Loss
- Kullback–Leibler Divergence (KL Divergence)
- ...

We only briefly discuss some

Mean Squared Error

Sum of the (squared) errors over all the examples in the train set (or test set if evaluating)

$$MSE = \frac{1}{m}\sum_{i}(\hat{y} - y)_i^2$$

- Use case: regression tasks
- Penalizes large errors more than smaller ones (the squared term)

Binary Cross-Entropy Loss

Compares the predicted probability of an event (ŷ and 1 - ŷ) with the real probability of the event, and penalizes the error logarithmically

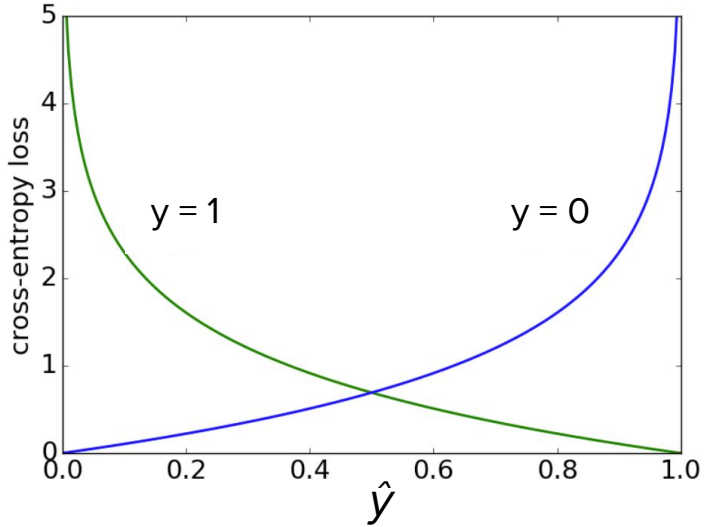
$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- Use case: binary classification tasks
- Paired with sigmoid on the output layer to normalize to [0, 1]

Logistic Regression: Cross-Entropy Loss

Intuition behind cross-entropy loss

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$



Cross-Entropy Loss

Similar to BCE, extended for the multi-class setting

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

- Use case: multi-class classification tasks
- Expects a probability distribution as input, typically paired with softmax activation output layer